

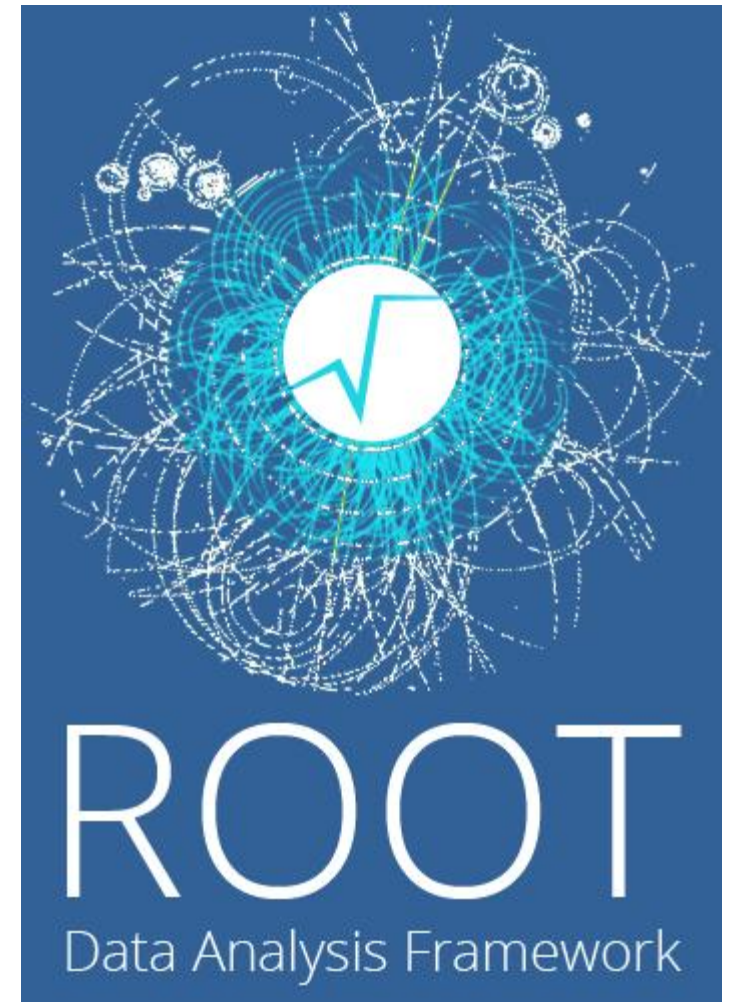
STFC – Nuclear Physics Summer School ROOT workshop

Aberdeen University

30/31st July 2026

Simon Gardner (University of Glasgow)

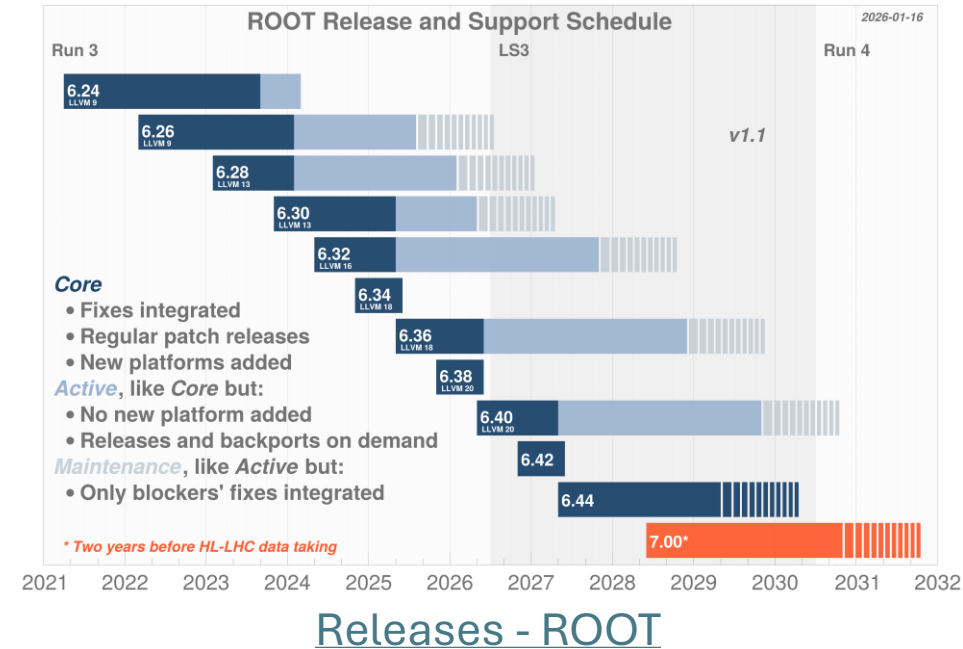
Simon.Gardner@glasgow.ac.uk



Introduction

- Two sessions for 2 hours each 13:00-15:00 Thursday/Friday
- Thursday
 - Exploring ROOT data files
 - RDataFrame
- Friday
 - Toy MC sample generation
 - Background subtraction

Activities are presented in C++ but if you're keen feel free to convert them to python
Feel free to ask questions at any point.



Caveats

Naturally, I don't know everything and am always keen to learn so if you have any alternative approaches to the exercises, please share!

ROOT is undergoing constant updates but is currently in a period of larger overhaul in preparation for ROOT 7.00. I intend to demonstrate best modern/future practices rather than entrenching legacy habits. I am guilty of some of these habits so some of this is new to me.

Warning – AI will be variably useful when trying to help with recent features.

Installation Notes

- With modern working practices it will rarely be up to the user to install ROOT.
- Instead, versions are available
 - through [CVMFS](#) – Will download libraries as required.
 - Docker/apptainer containers provided by larger collaborations.
- E.g. run/add to bashrc
 - `source /cvmfs/sft.cern.ch/lcg/app/releases/ROOT/6.38.06/x86_64-ubuntu22.04-gcc114-opt/bin/thisroot.sh`
- If you are contributing to ROOT, needing a non-standard installation or not connected to the internet installation may still be required.

Getting started

- Ensure you have access to a machine with ROOT
- Clone git repository locally:
 - `git clone git@github.com:simonge/STFC-NP\_Summer-School-2026\_ROOT.git`
 - (of git clone https://github.com/simonge/STFC-NP_Summer-School-2026_ROOT.git)
- Run the scripts:
 - `root MakeRNTuple.C`
 - `root MakeTTree.C`



You should have created two files one with the next generation RNTuple storage and one with the legacy TTree. You will likely need to deal with both in the future.

- RNTuple has both faster access and smaller file size
- Even with our tine test dataset
 - TTree script - 1.2M file, 0.06s
 - RNTuple script – 0.9M file, 0.03s
- (Not saying other details might be influencing the timing)
- Interface to the RNTuple requires more precise language.

Note: The dataset created has no physical meaning or correctness

Exploring ROOT files

Open your root files with

```
root [filename]
```

View the top level file content with

```
.ls
```

Exit root with

```
.q
```

When you run root with a script/datafile it will identify whether to open it as a file or execute a function in the script. ROOT stores and shows you a history so you can see what command was called on the file.

RNTuple file

Load as an RDataFrame:

```
ROOT::RDataFrame df("events","RNTupleFile.root")
```

Display the contents:

```
df.Display()->Print()
```

I imagine this may simplify to a single command in the future.

TTree file

TTree::Scan() – Method for directly looking at the events values in the tree.

In the root command line call:

```
events->Scan()
```

5 minute activity: Explore the contents of the file with the command line – Use the ROOT documentation to try and filter the output to show events with only two particle

Visually Exploring ROOT files

ROOT provides several interfaces to both local and remote files.

- Legacy Root TBrowser – when starting root pass `--web=off`
- Root7 RBrowser
- jsROOT

Legacy Root Browser

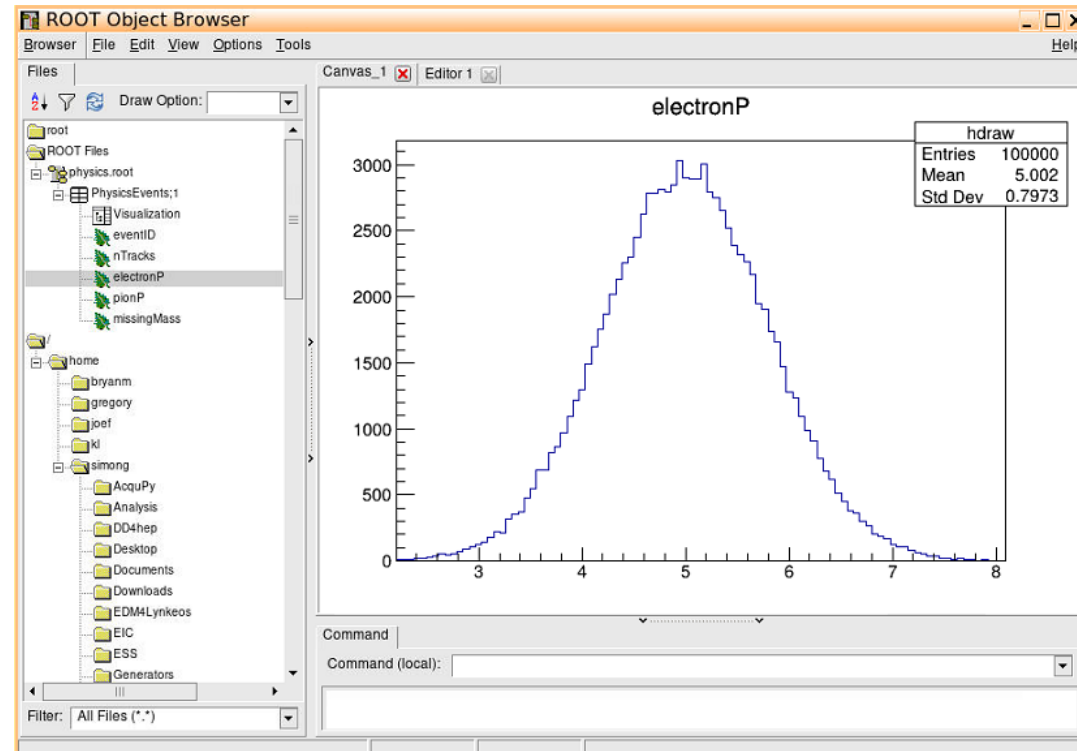
In your shell run

```
root --web=off TTreeFile.root
```

then in the root command prompt either of

```
TBrowser b  
new TBrowser
```

Opens ROOTs own gui



Visually Exploring ROOT files

ROOT provides several interfaces to both local and remote files.

- Legacy Root TBrowser – when starting root pass `--web=off`
- Root7 RBrowser
- jsROOT

Root7 Rbrowser (web browser based interface)

In your shell run

```
root physics.root
```

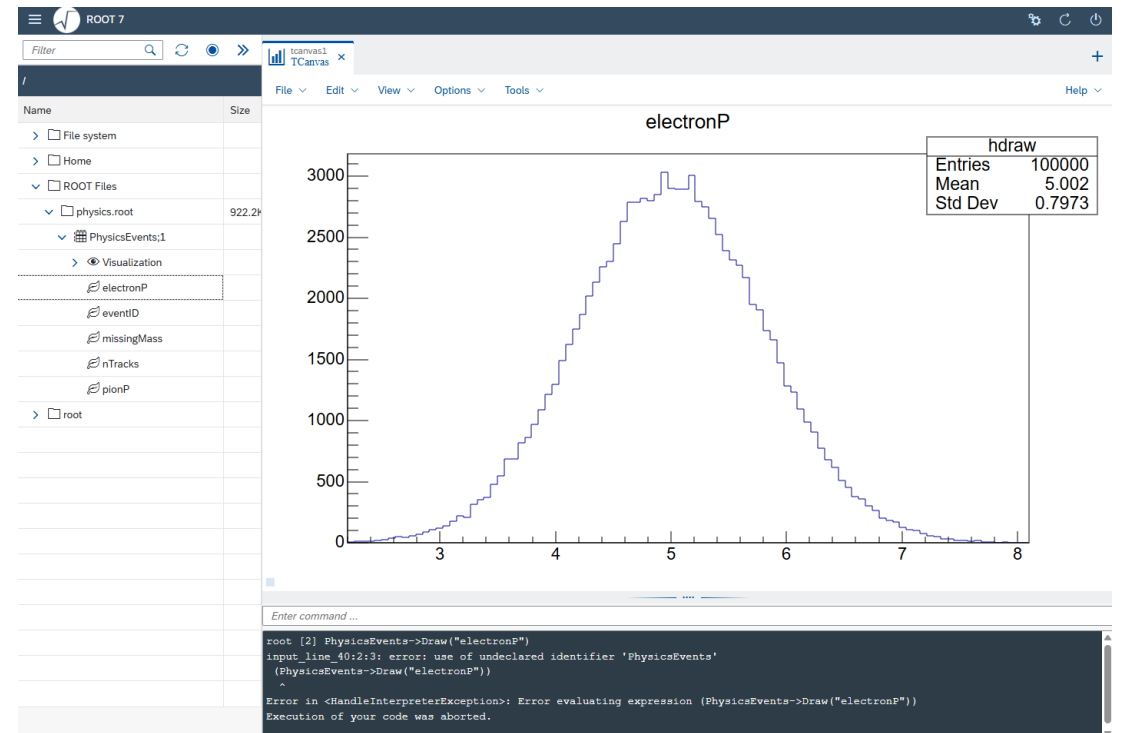
then in the root command prompt either of

```
TBrowser b  
new TBrowser
```

VSCode extension **ROOT RBrowser** available

Contains own command line interface

Previously was very slow so I had turned it off but appears to now be a good replacement.



Visually Exploring ROOT files

ROOT provides several interfaces to both local and remote files.

- Legacy Root TBrowser – when starting root pass `--web=off`
- Root7 RBrowser
- jsROOT

jsROOT

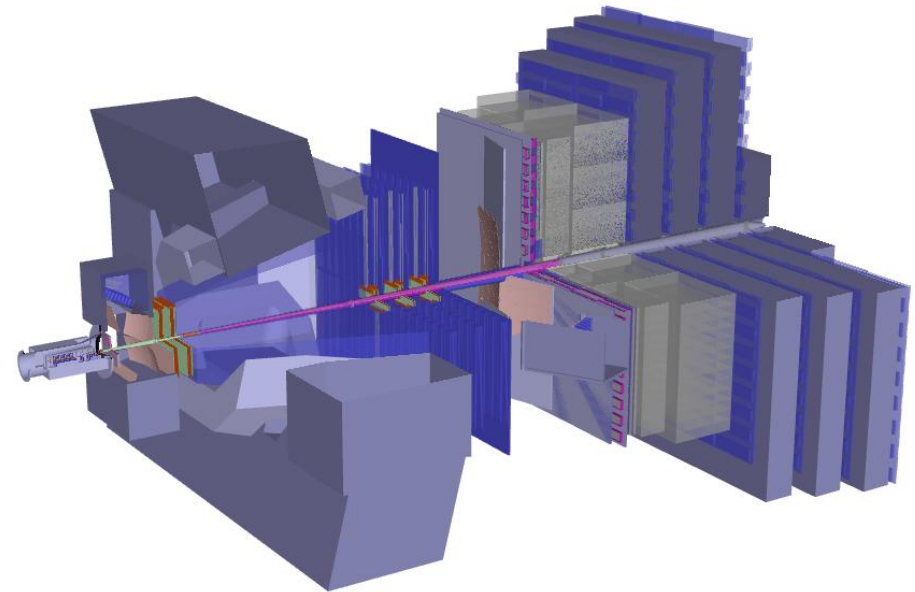
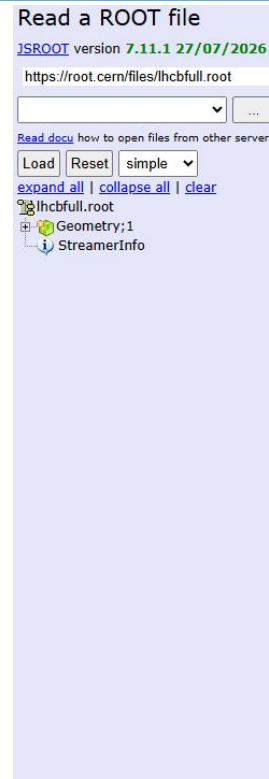
Visit <https://root.cern.ch/js/latest/> - version hosted by cern

Provides visualization of branches, histograms and geometry but no command line interface.

Very nice for visualization of root geometries
e.g. right - lhcb

VSCoDe has a **ROOT File Viewer** extension which runs an instance in VSCoDe

Doesn't currently appear to support RNTuples



Visually Exploring ROOT files

ROOT provides several interfaces to both local and remote files.

- Legacy Root TBrowser – when starting root pass `--web=off`
- Root7 RBrowser
- jsROOT

Drawing histograms from the command line

- TTree::Draw()
 - Very powerful single line allowing binning, filtering and visualization.
- RDataFrame::Histo1D followed by TH1D::Draw
 - The intermediate storage seems to be necessary at the moment. Interface is more transparent and explicit.

```
ROOT::RDataFrame df("events","RNTupleFile.root")
auto hist =
df.Histo1D({"nParticle","nParticles",10,0,10},"nParticles")
hist->Draw()
```

10 minute activity: Open your root file with each of GUIs.

Explore the tree provided.

Explore drawing histograms from the command line

Extra: Locate a public root file containing a detector geometry and explore it.

Running root code

- Root provides libraries which can be imbedded in larger code bases
- Often it is sufficient to run a script called through `root [script]`.
- This will call `.x [script]` once root is loaded.
- To add arguments when running from the command line you can do e.g.
 - `root MakeRNTuple.C\ (1000,\"RNTupleFile1000.root\"\\)`
 - Where “ and (are special characters in bash so need the \
 - This allows easy calling of the code multiple times through wrapper scripts.
- Calling root with a `+` after the script name asks it to compile it before running a `++` will force a recompilation event if one already exists.
 - `root MakeRNTuple.C+`
 - `root MakeRNTuple.C++`

RDataFrames

- ROOT::RDataFrames provides a similar interface to data python pandas dataframes.
- Greatly simplifies most analysis code – no manual looping through data
- Seamless multithreading: ROOT::EnableImplicitMT()
- Builds computation graph through Lazy execution.
 - Only loops over data once
 - Only performs calculations necessary to produce the outputs requested.

Ways of accessing ROOT data you may encounter

Manual address setting and index based looping

```
TFile f("file.root");
TTree *t = (TTree*)f.Get("myTree");

float px, py;
t->SetBranchAddresses("px", &px);
t->SetBranchAddresses("py", &py);

Long64_t n = t->GetEntries();
for (Long64_t i = 0; i < n; ++i) {
    t->GetEntry(i);
    float p = std::sqrt(px*px + py*py);
    if (p > 1.0) std::cout << p << "\n";
}
```

TTreeReader interface and iterator looping

```
TFile f("file.root");
TTreeReader reader("myTree", &f);

TTreeReaderValue<float> px(reader, "px");
TTreeReaderValue<float> py(reader, "py");

while (reader.Next()) {
    float p = std::sqrt((*px)*(*py));
    if (p > 1.0) std::cout << p << "\n";
}
```

RDataframe

```
ROOT::RDataFrame df("myTree", "file.root");

auto df2 = df.Define("p", "sqrt(px*px + py*py)")
              .Filter("p > 1.0");

df2.Display()->Print();
```

RDataFrames - [ROOT: ROOT::RDataFrame Class Reference](#)

```
ROOT::RDataFrame df("myTree", "file.root");

auto df2 = df.Define("p", "sqrt(px*px + py*py)")
             .Filter("p > 1.0");

df2.Display()->Print();
```

RDataFrame transformations we will use:

- Define – Attaches a new column to the dataframe.
- Filter – Identifies rows which match a criteria.

Transformations return a new node in the computational graph.

Can be chained together or returned to variables.

RDataFrame aggregation actions we will use:

- Histo1D,Histo2D,Graph – Creation of ROOT histograms/graphs
- Count,Max,Min,Mean,StdDev... – Calculation of column statistics

Calling these methods will book actions to the graph, not running anything until one of the outputs is requested

There are more generalized methods which can do arbitrary aggregation/reduction of the dataframe.

Saving RDataFrame columns to file

- Snapshot – Can choose columns you want to save to the file and the format.

Runs immediately

RDataFrames - Define

Define creates new columns of values in your dataframe.
It is useful to break long calculations down into small reusable components.
There are several useful ways this can be done.

Simple:

Define(New column name, Calculation as string)

```
auto newdf = df.Define("doubleNParticle", "nParticles*2")
```

Inline any column name will be interpreted from the string.

Functional:

For more complicated calculation you might need to use an external function to define your new column, especially if requiring a loop.

Define(New column name, Function which calculates values, input list of column names)

Lambda function [capture](arguments){algorithm}

Other functions or functors defined in your code are usable. Often a simple alternative is possible with ROOT/C++ vector operators.

```
.Define("chargeSumLambda",  
  [](const ROOT::RVec<int>& charges) { int sum = 0;  
    for (int q : charges)  
      sum += q;  
    return sum;  
  },  
  {"particleCharge"})
```

```
.Define("chargeSum", "std::accumulate(particleCharge.begin(), particleCharge.end(), 0)");
```

Warning: Due to the nature of the RDataFrame interface, error messages can be more difficult to debug

RDataFrames - Filter

Creates a filter which selects only certain rows from your data.

Very similar interface to Define.

Note - this does not filter on vectors in a column, a separate define is needed.

Simple:

```
Filter(Calculation which returns bool)
```

```
auto filteredDF = df.Filter("nParticles>2")
```

Inline any column name will be interpreted from the string.

Functional:

A more complicated filter can again use any function which returns a bool.

```
Filter(Function which calculates values, input list of column names)
```

Again, often standard vector operations can replace what you might be trying to achieve

RDataFrames – Histo1D, Histo2D

Creates a RResultPtr containing a standard ROOT histogram of the desired type

Will only actually try and fill the histogram when it is requested

Create histogram in RDataframe

```
auto hCharge =  
  selected.Histo1D(  
    {"hCharge",  
     "Charge Sum;#Sigma Q;Events",  
     22,  
     -11.0,  
     11.0},  
    "chargeSum");
```

Make sure you are using the RDataFrame node which contains the computation you require.

Standard histogram parameters, defining name, titles, bin widths.

Fill with the column “chargeSum”

Full event loop is only internally run when hCharge is needed, e.g.

hCharge->Draw()

hCharge->Write()

Previous TTree filling in multiple steps

```
TH1F *hCharge = new TH1F("hCharge",  
                          "Charge Sum;#Sigma Q;Events",  
                          22, -11.0, 11.0);  
  
while (reader.Next()) {  
  hCharge->Fill(*chargeSum);  
}
```

Warning: Due to the nature of the RDataFrame interface, error messages can be more difficult to debug

RDataFrames – Snapshot

Snapshot writes a dataframe columns to a ROOT file, either as a TTree or RNTuple

```
RResultPtr< RInterface< RLoopManager > > ROOT::RDF::RInterface< Proxied >::Snapshot ( std::string_view      treename,  
                                                                                   std::string_view      filename,  
                                                                                   const ColumnNames_t & columnList,  
                                                                                   const RSnapshotOptions & options = RSnapshotOptions() )
```

```
//Create snapshot of the filtered data frame  
ROOT::RDF::RSnapshotOptions opts;  
opts.fOutputFormat = ROOT::RDF::ESnapshotOutputFormat::kRNTuple;  
selected.Snapshot("events", outFilename,  
                  {"vertexR", "chargeSum", "nParticles"},opts);
```

Note- By default the snapshot will create a new file, the RSnapshotOptions need to be told to update

RDataFrame - computational graph

Aside - You can visualize the computational graph from RDF objects

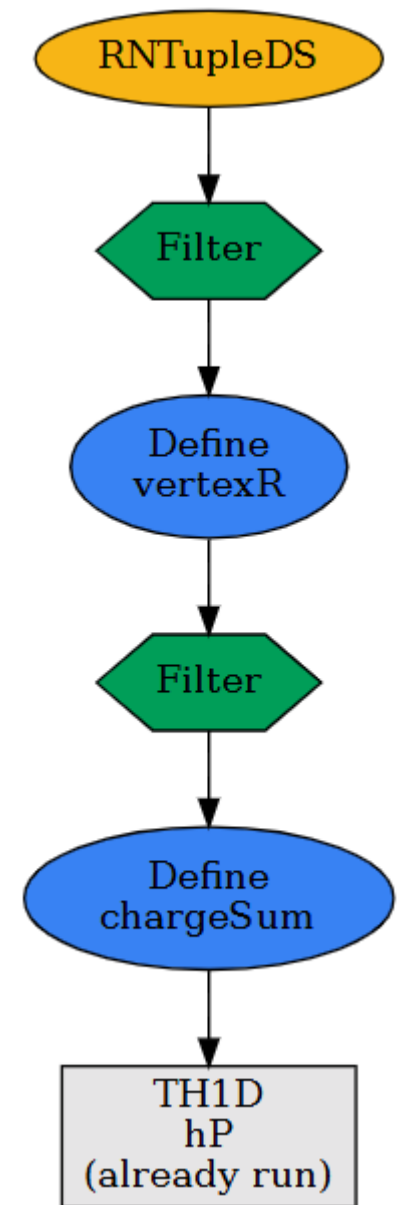
e.g.

```
// Save the computational graph to the dot file  
ROOT::RDF::SaveGraph(hP, "computational_graph.dot");
```

The .dot file is ascii but the dot command can convert it to a pdf

```
dot -Tpdf computational_graph.dot -o rdf_graph.pdf
```

The very simple graph showing what is needed to produce our hP ChargeSum histogram is shown here



Task – 1 hour?

- Run the Analysis.C macro
 - Explore the output root, pdf and dot files
 - Find how they are being created in the code
- Create a new analysis macro from scratch which
 - Creates histograms of
 - number of electrons, pions and protons in each event
 - The sum of the transverse momentum components for each event
 - A plot showing the correlation between the vertex z and number of particle.
- Suggested studies:
 - Select well-reconstructed events ($\text{vertexR} < 0.05$) and investigate whether the total reconstructed energy increases with particle multiplicity (nParticles vs visibleEnergy).
 - Select events containing at least one proton ($\text{nProtons} > 0$) and investigate how the interaction vertex position affects the number of reconstructed particles (vertexZ vs nParticles).
 - Select charge-balanced events ($\text{abs(chargeSum)} \leq 1$) and compare their missing transverse momentum distribution to that of charge-imbalanced events (missingPt).

I won't discourage using AI to approach these tasks, please ensure you read and understand the code you are running